University of Padova

Department of Mathematics "Tullio Levi-Civita"

Bachelor's Degree in Computer Science

Explainable Machine Learning through Large Language Models: Analysis and Prompt Design

Bachelor's Thesis

Supervisor

Prof. Michele Scquizzato

Undergraduate

Lorenzo Soligo

ID number 2101057

ACADEMIC YEAR 2025-2026

Lorenzo Soligo: *Explainable Machine Learning through Large Language Models: Analysis and Prompt Design*, Bachelor's Thesis, © June 2026

TO DO

—— TO DO

Dedicato a ...

Abstract

As Machine Learning (ML) models become increasingly prevalent in business applications, ensuring their transparency and reliability through Explainable Artificial Intelligence (XAI) is critical. This project, conducted in collaboration with Zucchetti S.p.A., investigates the interpretability and comprehensibility of various machine learning algorithms.

The research systematically evaluates a spectrum of predictive models, ranging from fundamental Regression and Classification models—such as Linear Regression, Logistic Regression, Support Vector Machines, and Decision Trees—to complex ensemble methods, including Random Forest, XGBoost, RuleFit, and Symbolic Regression. The analysis explores both intrinsic model transparency and post-hoc explainability techniques, utilizing metrics like SHapley Additive exPlanations (SHAP) to interpret predictions and feature importance. Furthermore, the methodology encompasses the examination of mathematical foundations, internal knowledge representations, and the impact of ensemble techniques on both model performance and explainability. Real-world validation is performed using datasets such as Student Salary Prediction.

A primary objective of this work is bridging the gap between technical accuracy and human-understandable explanations for non-expert stakeholders. To achieve this, the project leverages advanced Prompt Engineering principles. Tailored prompts for Large Language Model (LLM) are developed for each analyzed algorithm. These prompts are explicitly designed to automatically generate human-readable explanations of algorithmic behaviors and predictions. Ultimately, the project delivers production-ready implementations and LLM integration adapters , demonstrating how the synthesis of traditional XAI methodologies with modern language models can significantly enhance the transparency, trust, and responsible deployment of AI systems.

TO DO

—— TO DO

Acknowledgements

TO DO Prof. Michele Scquizzato .

TO DO

TO DO

Padova, June 2026 *Lorenzo Soligo*

Contents

List of Figures

List of Tables

Chapter 1

Introduction

Introduction to the stage idea, goals, company and thesis organization. This chapter is meant to provide the necessary context for the reader to understand the motivation and the structure of the thesis.s

1.1 Company

Zucchetti S.p.A. is a leading Italian software company specializing in providing comprehensive IT solutions for businesses. Founded in 1978, Zucchetti has grown to become one of the largest software providers in Italy, offering a wide range of products and services that cater to various industries, including manufacturing, retail, healthcare, and public administration.

With a strong focus on innovation and customer satisfaction, Zucchetti is committed to helping organizations optimize their operations and achieve their business goals through cutting-edge technology. Exactly from this commit, the company has been actively involved in the development and implementation of ML and Artificial Intelligence (AI) solutions, aiming to enhance the capabilities of their software offerings and provide more value to their clients.

Zucchetti S.p.A. logo.

1.2 Stage idea

The internship project is centered around the exploration of XAI techniques, with a specific focus on the interpretability and comprehensibility of machine learning algorithms. The primary objective is to analyze a variety of predictive models, ranging from basic regression and classification algorithms to more complex ensemble methods, in order to evaluate their transparency and explainability. The idea is to increase the undestanding of the results of the models, and to make them more accessible to non-expert stakeholders, by leveraging advanced Prompt Engineering (PE) principles to develop tailored prompts for LLMs that can automatically generate human-readable explanations of algorithmic behaviors and predictions.

The timeline of the project is structured as follows:

Week	Task
1st	Analysis of Linear Regression, Logistic Regression, Support Vector Machines.
2nd	Analysis of Decision Trees, Random Forests, XGBoost.
3rd	Code implementation and Prompt Engineering for the analyzed algorithms.
4th	Evaluation and documentation of the results.
5th	Analysis of RuleFit and deep dive in Random Forests.
6th	Analysis of Symbolic Regression.
7th	Code implementation and Prompt Engineering for the analyzed algorithms.
8th	Evaluation and documentation of the results, final report writing.

Table of the project timeline.

1.3 Project goals

The goals follow a notation to distinguish between necessary (N), desirable (D) and optional (O) goals. In the planning phase the goals were defined as follows:

Goal	Description
O-01	Complete and profound comprehension of the choosen algorithms, their mathematical foundations and their internal knowledge representation.
O-02	Comprension of interpretability and explainability techniques in machine learning.
O-03	Comprehension of algorithms design techniques.
O-04	Prompt generation for Large Language Models to generate human-readable explanations.
O-05	Comprehension and application of Prompt Engineering principles in the context of XAI.
D-01	Creation of a metric to evaluate the explainability of the analyzed algorithms.
D-02	Automatization of the analysis pipeline from dataset to LLM requests.
F-01	Application in real-world scenarios of the interpretability and explainability of Machine Learning models.
F-02	Explainability study of semi-supervised and unsupervised algorithms.

Table of project goals.

1.4 Thesis structure

The second chapter describes the basic concepts that fuel the project. The concepts spread from the mathematical foundations of the analyzed algorithms, the basics of explainability, to the principles of Prompt Engineering and LLMs.

The third chapter analyzes the ML algorithms, focusing on their capability, limitation and interpretability.

The fourth chapter describes the design and implementation of the analysis pipeline from dataset to LLM requests.

The fifth chapter discusses prompt engineering techniques and their application in the context of XAI.

The sixth chapter presents the conclusions of the study, the findings and the goals assessment.

Chapter 2

Background knowledge

In this chapter, we will provide the necessary background knowledge to understand the project. We will cover the mathematical foundations of the analyzed algorithms, the basics of explainability, and the principles of Prompt Engineering and LLMs.

2.1 Algorithmic analysis structure

The analysis of the algorithms is structured in a systematic way to ensure a comprehensive evaluation of their interpretability and comprehensibility as well as thir functioning. The structure includes the following components:

1. **Mathematical foundations**: detailed examination of the mathematical principles underlying each algorithm, including their assumptions and theoretical properties. The focus is on the understanding of the specific Objective Function that the algorithms optimize, the regularization techniques they use, and the mathematical properties that govern their behavior.
2. **Computational complexity**: analysis of the computational requirements of each algorithm, including time complexitys and scalability reguarding both training and inference.
3. **Internal representation**: exploration of how each algorithm internally represents data and makes decisions, especially regarding Categorical Features.
4. **Data assumptions**: investigation into the assumptions each algorithm makes about the data, such as linearity, independence of features, and distributional assumptions.
5. **Predictive performance and limitations**: evaluation of the predictive performance of each algorithm on relevant datasets, considering the data assumptions and mathematical foundation of the model.
6. **Metrics for prediction quality**: presentation of the most relevant metrics for evaluating the models performance, based on the task.
7. **Metrics for interpretability**: presentation of the most relevant metrics for evaluating the models interpretability, based on the task.
8. **Explainability limitations**: discussion of the limitations of explainability techniques for each algorithm, including potential pitfalls and challenges in interpreting their predictions.

2.2 Explainability

Explainability (or interpretability) of a ML model is the ability to communicate **how and why** the model made a prediction.
Contrary to intuition, explainbility it's **not a binary attribute**, but a spectrum with multiple dimensions.

2.2.1 Explainbility dimensions

1. **Intrinsic Transparency (Model-Level Interpretability)**
   The model is inherently transparent and interpretable by design. The structure directly communicates how it works, such as regression coefficients or decision tree nodes. Each prediction is fully traceable, and parameters have tangible meanings. Explanations reflect human reasoning, making them coherent from a domain perspective.
   It rappresents the best case scenario for explainability but it is often in trade-off with performance, as more complex models tend to be less transparent but more powerful. It is generally organized in tears: High, Medium and Low trasparency.
2. **Global Interpretability (Model-Level Explainability)**
   The model is comprehansible in its entirety, allowing to understand its overall behaviour and decision-making process. Such a level of understandsing it's hardly achievable and it is usually undirectly reached by comprehending the singular components of the models instead of the model as a whole.
3. **Modular Interpretability** When global interpretability is not achievable, we can still understand the model by analyzing its components separately. Exploiting the model transparency, we can understand the role of each component in the decision-making process.
   This concept is clear in the case of linear methods, where it is easy to understand the contribution of the single weights or in tree based models, where the splits and the structure of the tree can be analyzed separately.
4. **Local Interpretability (Instance-Level Explainability)** The model might be opaque and to complex to understand globally. However, for a specific prediction, we can generate an explanation that is locally interpretable. SHAP could be used in this context to explain a particular instance obtained a certain prediction.

2.2.2 Explanation properties and factors that facilitate understanding

For human explanations to be effective, they should have certain properties that facilitate understanding and usability. The following properties can be identified from the work of Robnik-Sikonja and Bohanec 2018 and Molnar [1]:

- **Contrastive explanations**

Comparative explanations that highlight the differences between instances can be more informative than absolute explanations. The instances confrontation makes the explanation more impactful, providing a direct idea of how different factors contribute to different outcomes.s

- **Cause selection**

Not all features are equally important; explanations should focus on the most influential factors rather than listing all contributing features. A small sample of the most relevant features provide a more clear explanation than a long list of all features prevventing information overload.

- **Social and contextual factors**

The social environment and the audience's expertise level should be considered when generating explanations to ensure they are appropriate and understandables.

- **Anomalies or abnormal predictions**

Highlighting anomalies or abnormal predictions can help users understand when the model is making an unusual decision, which can be crucial for trust and debugging. Features that are not largerly relevant for the majority of the predictions but that have a large impact on specific predictions should be highlighted for their capability.

- **Fidelity**

The explanation should accurately reflect the model's behavior and not introduce misleading information. A high-fidelity explanation ensures that users can trust the insights provided and make informed decisions based on them, provided that the model is accurate in its predictions.

- **General and probable**

In the absence of anomalies of specific cases, a good explanation should rely on general and probable causes that are likely to be true for most instances. This parameter can be quantified by the feature support:

$$\text{Feature Support}_i = \frac{\#\text{training instances with similar values to } \ x_i}{\#\text{total training instances}}$$

Generality can be misleading, as it may lead to explanations that are too broad and not specific enough to be useful. It's important to balance generality with the impact of the explanation, ensuring that it provides meaningful insights without being overly vague. This is why usually abnormal features are highlighted, as they can provide more specific and impactful explanations.

2.2.3 SHAP analysis

SHAP (SHapley Additive exPlanations) is a method for explaining the output of machine learning models by attributing the prediction to each feature. It is based on the concept of Shapley values from cooperative game theory, which provide a way to fairly distribute the "payout" (in this case, the prediction) among the "players" (the features) based on their contribution to the prediction. SHAP values can be used to understand the importance of each feature for a specific prediction (local interpretability) or for the model as a whole (global interpretability). SHAP provides a unified framework for interpreting various types of models and can be applied to both linear and non-linear models, making it a powerful tool for explainable AI.

SHAP library logo.

2.3 Prompt Engineering and LLMs

The following sections will present the principles used in the design of the prompts for the LLMs to generate human-readable explanations of the analyzed algorithms.

2.3.1 Expert Personas

The expert personas techiniques consists in the design of prompts that describes the role that the LLM should play in the task. This technique first should guide the model in generating a more precize, coherent and relevant output.

Though the expert personas seems to be linked to a loss of accuracy in multiple benchmarks in the recent studys by Zizhao Hu [2], the principal use of the LLMs in this project is results evaluation and text generation. The data and metrics do not have to be generated and have only to be read directly from the prompt. The paper specifically talks about *"For tasks that depend on pretrained knowledge retrieval accuracy"* [2]. It's reasonable to think that in such a context, loss of accuracy is not a critical issue. In the same paper, an alignment of behavior and style to the descripted personas is observed. This drift in behavior is critical in the task of describing the results of an algorithm in a human-readable way, as the style of the explanation is fundamental to make it accessible to non-expert stakeholders.

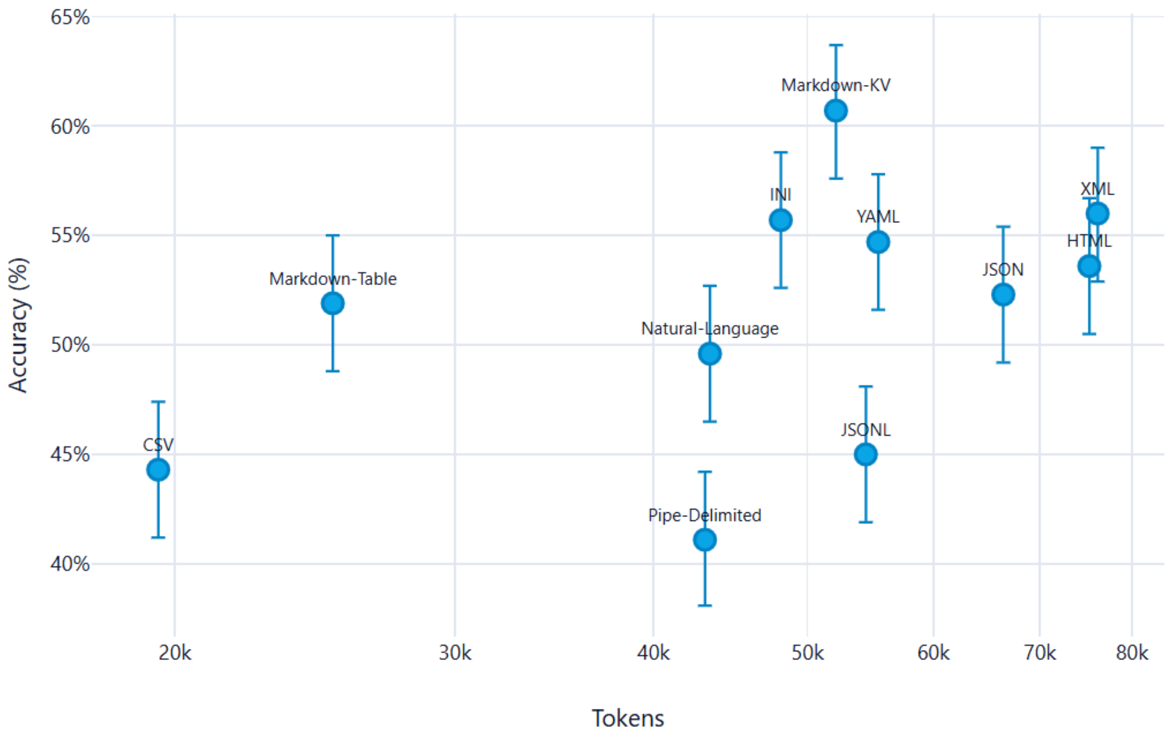
2.3.2 Familiar formats

The extensive use of LLMs have led to the emergence of some familiar formats that are widely used in the design of prompts.

The principal features that this formats present, and that makes them particularly effective, are the following:

- Widely use in training: the more a format is used in the training of the model, the more likely is to be effective in the interpretation of content in that format.
- Clear structure: the format should have a clear and recognizable structure that allows the model to easily identify the different components of the prompt and their relationships.
- Token efficiency: the format should be designed in a way that minimizes the number of tokens used, while still conveying all the necessary information. This is particularly important in the context of LLMs, as the number of tokens used can have a significant impact on the performance and cost of the model.

From the multitude of formats in existence, some emerged for their particular accuracy and efficiency. The most valueable one is **Markdown**, which has a clear and recognizable structure that allows the model to easily identify the different components of the prompt and their relationships. The use of markdown also allows to create a clear and organized structure for the generated explanations, making them more readable and accessible to non-expert stakeholders. Another accurate format is YAML, which offers a more readable formatting than JSON and XML while still being well structured.
An alternative like CSV can be used, as it is a widely used format for tabular data and is likely to be well understood by the model. However, it's important to ensure that the CSV format is properly structured and that the necessary information is included in the prompt to allow the model to generate accurate and relevant explanations.
What follows is a summarizing image whe re the 11 formats taken in exam in the study[3] are presented. The y-axis is a scale of Accuracy while the x-axis a token consumption scale. It's clear that in this particular case of study, only OpenAI's GPT-4.1 nano has been tested, Markdown emerges as the most accurate format, while mantaining good token usage.



Formats for prompting. Source: @formats-llms.

2.3.3 Chain-of-Thought

Chain of Thought (CoT) prompting is a technique that consists in the design of prompts that guide the LLMs to generate a step-by-step reasoning process to reach the final answer. This technique is particularly useful in the context of this project, as it allows to generate explanations that are more coherent and relevant to the analyzed algorithms.

The additional value of th guided, step-by-step reasoning has been proven in multiple benchmarks[4], and it is particularly useful in the context of this project, as it allows to generate explanations that are more coherent and relevant to the analyzed algorithms.

In the particular context of XAI and non-technical audiences because it enhances the model's ability to provide detailed and understandable explanations. The generated answer is not just a final result, but a comprehensive explanation that breaks down the reasoning process into clear and logical steps.

2.3.4 Zero-Shot prompting

This technique consists in the design of prompts that do not include any example of the expected output. It's the simplest kind of prompting because it erases the problem of having to provide training examples. Such method is particularly necessary in the context due to the fact that an example of datas and metrics for reference could be to specific and not generalized enought. An additional set of datas could also confuse the model and make it less accurate or biased on

the actual result of the analysis.
The zero-shot prompting also revealed to be quite effective with some additional techniques such as the use of Chain of Thought (CoT) prompting [5].

2.3.5 Multimodal prompt

The multimodal prompting technique consists in the design of prompts that include multiple modalities, such as text, images, tables, etc. This technique is particularly useful in the context of this project, as it allows to provide the model with a richer and more comprehensive context for generating explanations.
The use of multimodal prompts can enhance the model's ability to comprehend and analyze the results of the algorithms, as it can leverage the information provided in different formats to generate more accurate and relevant explanations. In fact, *"visual prompts effectively interact with the textual prompts, enhancing the alignment between modalities and thereby improving the model's performance on the zero-shot instruction learning"* [6].

Chapter 3

ML algorithms analysis

In this chapter, we will analyze the machine learning algorithms used in the project, looking at the mathematical foundations, the data requirements, the predictive performance, and the interpretability of the results.
This initial analysis will guide the design and implementation of the pipeline as well as the creation of the specific prompts for the LLMs to generate human-readable explanations of the results.

3.1 Linear regression

3.1.1 Mathematical model

The linear regression model is the simplest form of regression. It assumes a linear relationship between the input features and target variable. The mathematical representation of the linear regression model is:

$$y = \sum_{i=0} \beta_i x_i \forall i \in F$$

Where F is the set of features, β are the calculated weights for each of the features, and y is the target variable.
The goal of the linear regression is to find the optimal weights β that minimize the error between the predicted values and the actual target variable. This is typically done by minimizing the ordinary least squares loss function between the predicted values and the actual target variable. The optimization problem can be formulated as:

$$\hat{\beta} = \text{argmin}_{\beta_0 \ldots \beta_p} \sum_{i=1}^{n} (y^{(i)} - (\beta_0 + \sum_{j=1}^{p} \beta_j x_j))^2$$

3.1.2 Time complexity

Mainly there are two approches to solve this optimization problem: the analytical solution and the iterative optimization methods (e.g. Gradient Descent (GD)). The analytical solution is given by the normal equation:

$$\hat{\beta} = (X^T X)^{-1} X^T y$$

In this formulation, X is an nxp matrix where n is the number of instances and p is the number of features. The matrix X is augmented with a column of 1s to account for the intercept term β_0. The computational complexity of the analitical solution is dominated by

the matrix multiplication and inversion operations. In fact, the cost for the moltiplication of matrixes is $O(p^2n)$ and the cost for the inversion of a pxp matrix is $O(p^3)$. Therefore, the overall computational complexity of the analytical solution is $O(p^2n + p^3) = O(p^3)$, which for vast datasets, over 10000 records, becomes prohibitive.
The iterative methods, such as GD, calculate the gradient of the loss function with respect to the weights and update the weights iteratively until convergence. The computational complexity of the gradient calculation is $O(pn)$ per iteration, and the number of iterations required for convergence can vary depending on the learning rate and the specific dataset. However, in practice, iterative methods can be more efficient than the analytical solution for large datasets, as they do not require matrix inversion and can converge faster with appropriate hyperparameter tuning. Other methods like Stochastic Gradient Descent (SGD) can further reduce the computational cost by approximating the gradient using a subset of the data at each iteration. On the other side, for smaller dataset, analytical solution offers convergence in a single step, without the need for hyperparameter tuning, and guarantees a global optimal solution.

For **inference**, the computational complexity is $O(p)$ per instance, as it involves a simple dot product between the feature vector and the weight vector.

3.1.3 Spacial complexity

The spacial complexity of the linear regression model is determined by the need to store the input data, the weight vector, and any intermediate matrices used in the analytical solution. Specifically, the analytical solution requires storing the nxp matrix X, the pxp matrix X^TX, and the p-dimensional weight vector β. Therefore, the overall spacial complexity of the linear regression model is dominated by the storage of the input data and the intermediate matrices, resulting in a spacial complexity of $O(np + p^2)$.
For the GD method, the spacial complexity is reduced to $O(np + p)$, as it does not require storing the intermediate matrix X^TX.
For **inference**, the computational complexity is $O(p)$ per instance, as it only stores the weight vector and the feature vector.

3.1.4 Internal representation

Linear regression represents the resulting weight of the features as a vector of weights, $\beta = [\beta_0, \beta_1, ..., \beta_p]$.
Specific encoding is needed for categorical features, which are typically handled through one-hot encoding, where each category is represented as a binary feature. This can lead to an increase in the number of features and potential multicollinearity issues if not handled properly.
In regard of **explainability**, the internal representation of linear regression is straightforward and transparent, which makes it one of the most interpretable machine learning models. The weights directly indicate the strength and direction of the relationship between each feature

and the target variable. This allows for a clear understanding of how each feature contributes to the prediction, making it easier to communicate insights to stakeholders and identify important predictors in the data.

3.1.5 Data assumptions

Linear regression relies on several key assumptions about the data to ensure the validity of the model and the reliability of its predictions. These assumptions include as described by Molnar [1]:

1. **Linear constraints:** relationships between features and target must be linear. Other kind of relationships must be manually introduced and cannot be revealed automatically
2. **Residuals normality:** the residuals $\epsilon_i = y_i - \hat{y}_i$ must follow a normal distrubution. Violazioni gravi compromettono l'inferenza statistica (p-value, intervalli di confidenza)
3. **Homoscedasticity:** residuals must have constant variance across all levels of the features. In practice, this assumption is **frequently violated**. Example: in real estate, the price of very large houses is extremely variable, while that of small houses is concentrated.
4. **Indipendence of measurements:** instances don't have to be correlated. Dipendent datas, such as time series, violate this assumption and as such should not be investigated with linear regression.
5. **Feature fisse:** le feature devono essere considerate come costanti, non soggette a errori di misurazione significativi. Se le feature contengono errore di misura, i coefficienti sono distorti (attenuation bias)
6. **Absence of multicollinearity:** Feature should not be highly correlated. Multicollinearity causes numerical instability during the inversion of $X^T X$ e weights inflation in absolute value. Using GD the geometrical properties of the loss function changes and leads to very slim vallies causing an important reduction of the learning rate.

3.1.5.1 Preprocessing

The data assumption lead to specific preprocessing steps to determine the suitability of the data for linear regression and to improve the performance of the model. These steps include:

1. **Multicollinearity identification:** calculation of the Correlation Matrix between features using the Pearson's coefficient or VIF (Variance Inflation Factor) to identify highly correlated features. $\text{VIF}_j = \frac{1}{1-R_j^2}$

Where R_j^2 is the coefficient of determination for the regression of feature j against all other features. More on R_j^2 here.

3.1.6 Predictive performance and limitations

As discussed, the linear regression imposes several constraints on the data and model performance. Very good performances are achieved whenever linear relationships exist between features and target both in accuracy and efficiency.

However, in real-world scenarios, these conditions are often violated, leading to poor predictive performance. The model is particularly sensitive to outliers, which can disproportionately influence the weights and lead to skewed predictions. Additionally, linear regression is **not suitable for capturing complex, non-linear relationships** in the data, which limits its applicability in many real-world problems where such relationships are common. Finally, the model's performance can degrade significantly when the number of features is large relative to the number of observations, leading to overfitting and poor generalization to new data. This limitation is intensified by the presence of categorical features.

3.1.7 Pure metrics for prediction quality

What follows is a list of most relevant metrics for evaluating the predictive performance of linear regression models, based on the task and the data assumptions.

3.1.7.1 R^2 (Determination Coefficient)

R^2 quantifies how much the model explains the total variance of the data. Il ranges from 0 to 1, where 0 is a model that cannot explain the datas and 1 is a perfect adherance.

$$R^2 = 1 - \frac{SSE}{SST}$$

Where:

- $SSE = \sum_{i=1}^{n} (y^{(i)} - \hat{y}^{(i)})^2$ is the sum of squared residuals, quantifying the variance unexplained by the model
- $SST = \sum_{i=1}^{n} (y^{(i)} - \bar{y})^2$ is the total variance

3.1.7.2 $\bar{R}^2$ (Adjusted R²)

R^2 tends to increase with the number of features, even if those features are not useful. Adjusted R^2 penalizes the addition of non-informative features.

$$\bar{R}^2 = 1 - (1 - R^2)\frac{n-1}{n-p-1}$$

Where n is the number of instances and p is the number of features.

3.1.7.3 Feature Importance (t-statistic)

$t_{\hat{\beta}_j}$ measures the statistical significance of each coefficient, calculated as the weight scaled by its standard error:

$$t_{\hat{\beta}_j} = \frac{\hat{\beta}_j}{SE(\hat{\beta}_j)}$$

Intuitively, the higher the absolute value of a coefficient is, the more the feature is statistically significant in predicting the target variable.

As intuitively, the hgher the variance of the coefficient is, the less the feature is significant, as the model is more uncertain about the true value of the coefficient.

3.1.7.4 p-value

p-value is a measure of the probability of "obtaining the observed data under the null hypothesis of a statistical test"[7]. In the context of linear regression, the null hypothesis is that the true coefficient β_j is equal to zero, meaning that the feature does not have a significant impact on the target variable. The p-value for each coefficient is calculated based on the t-statistic and indicates the probability of observing such an extreme value for $t_{\hat{\beta}_j}$ if the null hypothesis were true. A common convention is to consider a p-value less than 0.05 as statistically significant, suggesting that there is strong evidence against the null hypothesis and that the feature likely has a meaningful relationship with the target variable.

3.1.7.5 Mallows' Cp

Mallows' Cp is a model selection metric that balances model fit and complexity, calculated as:

$$Cp = \frac{SSE}{\hat{\sigma}^2} - n + 2p$$

Where $\hat{\sigma}^2$ is the estimate of residuals variance of the complete model. It's used to reduce the problem of overfitting, reducing the number of features.

3.1.7.6 Root Mean Squared Error (RMSE)

RMSE measures the magnitude of the errors, in the same scale as the target variable:

$$\text{RMSE} = \sqrt{\frac{SSE}{n}}$$

3.1.7.7 Mean Absolute Error (MAE)

MAE measures the average magnitude of the errors, without considering their direction:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^{n} \mid y_i - \hat{y}_i \mid$$

3.1.8 Diagnostic plots

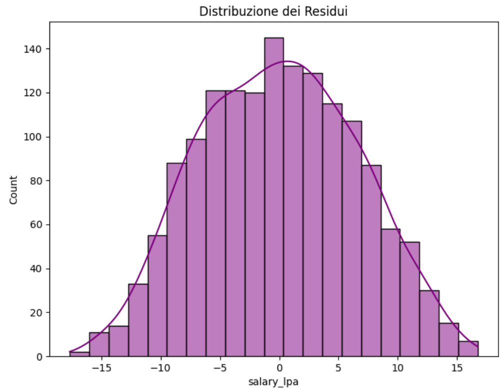
The use of diagnostic plots is crucial to visually assess the assumptions of linear regression and to identify potential issues with the model fit.

3.1.8.1 Actual vs Predicted

Scatter plot with actual values y_i on the y-axis and predicted values $\hat{y}_i$ on the x-axis. $\hat{y}_i$. If the model fits well, the points should be concentrated around the diagonal line $y = \hat{y}$. Deviations from this pattern can indicate various issues with the model fit.

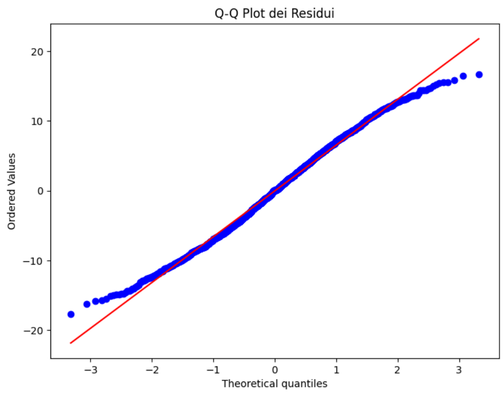
3.1.8.2 Histogram of residuals distribution

Distribution of the residuals $\epsilon_i = y_i - \hat{y}_i$. It's especially useful to identify violations of the normality assumption. A normal distribution of residuals is expected for valid inference, and deviations from this pattern can indicate issues with the model fit or the presence of outliers.



Histogram of residuals distribution example for linear regression.

3.1.8.3 Q-Q Plot (Quantile-Quantile)



Q-Q Plot of residuals example for linear regression.

Another way to check the normality of residuals is through a Q-Q plot, which compares the quantiles of the residuals to the quantiles of a normal distribution. If the residuals are normally distributed, the points in the Q-Q plot should approximately follow a straight line. Deviations from this line, especially at the tails, can indicate non-normality of the residuals, which may affect the validity of statistical inference based on the model.

3.1.8.4 Residuals vs Fitted Values

Scatter plot with fitted values $\hat{y}_i$ on the y-axis and residuals $\epsilon_i = y_i - \hat{y}_i$ on the x-axis. $\hat{y}_i$. If the model fits well, the points should be concentrated around the diagonal line $y = \hat{y}$. This plot can be used to identify violation in regression assumptions (Section 3.1.5). Increasing dispersion for example, show heteroschedasticity, while systematic patterns (e.g. a curve) indicate non-linearity that the model cannot capture.

3.1.9 Explainability and interpretability metrics

3.1.9.1 Feature Effect

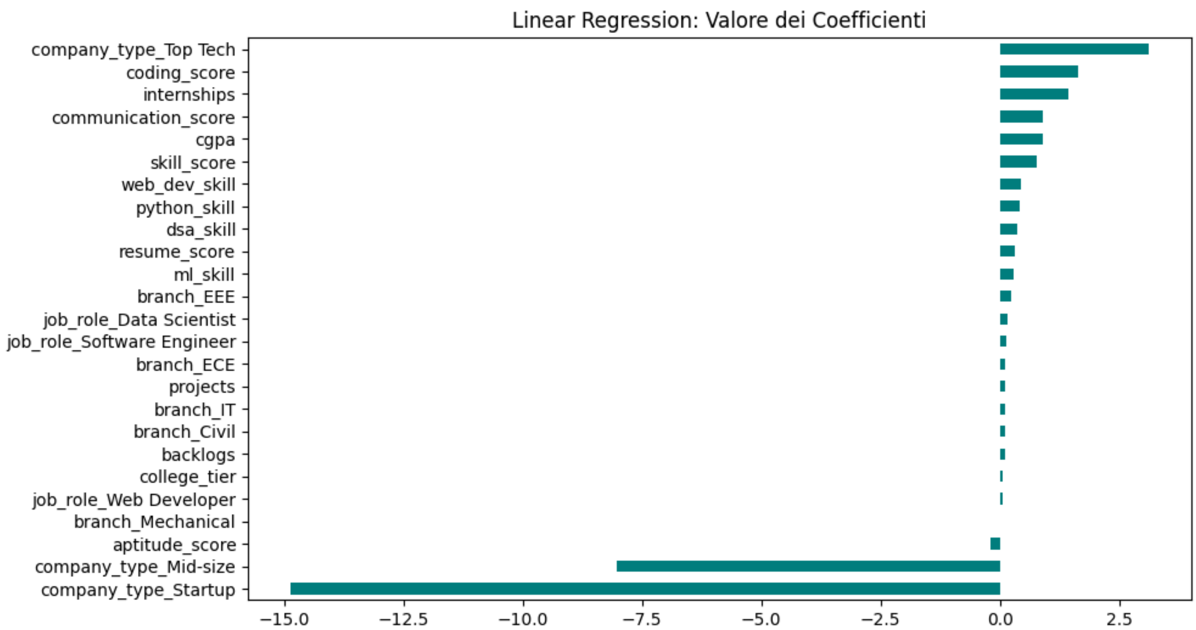
Linear regression naturally provides a direct measure of the feature effect on the target variable through the coefficients β_j. The effect of a feature x_j on the prediction for an instance i can be calculated as:

$$\text{effect}_j^{(i)} = \beta_j x_j^{(i)}$$

The standard visualization shows a box plot of the calculated effect in the quantiles 25, 50 and 75 for each feature. This allows to quickly identify the features with the most significant effect on the predictions, as well as the distribution of the effects across the dataset.
This plot results not useful if the datas are normalized, as the effect is calculated as the product of the coefficient and the feature value, and normalization can obscure the true impact of the features on the predictions.

3.1.9.2 Weight Plot

For a direct visualization of the importance of each feature, a weight plot can be used, which shows the coefficients β_j for each feature. The coefficients indicate the strength and direction of the relationship between each feature and the target variable.



Weight Plot of coefficients example for linear regression.

3.1.10 Explainability limitations

As emerged until now, linear regression is one of the most interpretable machine learning models due to its transparent internal representation and the direct relationship between features and predictions. The impact of each feature on the prediction can be easily understood through the coefficients, which indicate how much the prediction changes with a one-unit change in the feature, holding all other features constant.

What makes the model extremely interpretable make it limited in its predictive ability. Linearity of the relationships is as understandable as restrictive.

3.2 Logistic regression

3.2.1 Mathematical model

The logistic regression is a model mainly used for binary classification problems where the response variable is dichotomous. Similar to linear regression(Section 3.1) it uses a linear combination of the input features, but instead of directly outputting a continuous value, it applies a logistic function to map the output to a probability between 0 and 1.

$$\sigma(z) = \frac{1}{1 + \exp(-z)}$$

The output can consequently be interpreted as the probability that the input instance belongs to the positive class (y = 1). In particular, the probability that an instance belongs to the positive class is given by:

$$P(y^{(i)} = 1 \mid x^{(i)}) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^{p} \beta_j x_j^{(i)}))}$$

And the probability that it belongs to the negative class (y = 0) is $P(y^{(i)} = 0) = 1 - P(y^{(i)} = 1)$, following the Bernoulli distribution.

The goal of the logistic regression is to find the parameters β that best fit the data, which is done by maximizing the **likelihood** of observing the data given the parameters.

$$L(\beta; y, X) = \prod_{i=1}^{n} P(y^{(i)} = 1)^{y^{(i)}} \cdot (1 - P(y^{(i)} = 1))^{1-y^{(i)}}$$

For numerical stability **log-likelihood** is usually calculated:

$$\ell(\beta) = \sum_{i=1}^{n} \left[y^{(i)} \log(P(y^{(i)} = 1)) + (1 - y^{(i)}) \log(1 - P(y^{(i)} = 1)) \right]$$

Differently from linear regression, the logistic regression does not have a closed-form solution for the parameters that maximize the likelihood so only iterative optimization algorithms like GD are available

3.2.2 Time complexity

As described in Section 3.1.2, the time complexity of GD is $O(pn)$ per iteration, and the number of iterations required for convergence can vary depending on the learning rate and the specific dataset. For **inference** the time complexity is again, $O(p)$ per instance.

Since the optimization is iterative, the overall time complexity can be less predictable than linear regression, especially if the data has features that lead to slow convergence (e.g., highly correlated features or features that cause the decision boundary to be close to the data points). However, in practice, logistic regression is often computationally efficient for moderate-sized datasets and can be scaled to larger datasets using SGD or mini-batch gradient descent.

3.2.3 Spacial complexity

The model stores a vector of coefficients β of size p, which requires $O(p)$ memory. During training, additional memory is needed to store the intermediate values for the optimization algorithm, which can be $O(n)$ for batch gradient descent due to the need to compute the predictions for all instances in each iteration.
For inference, the memory complexity is $O(p)$ for storing the coefficients and $O(1)$ for the input instance, leading to an overall spacial complexity of $O(p)$.

3.2.4 Internal representation

The model internally rappresents the values as a vector of weights, $\beta = [\beta_0, \beta_1, ..., \beta_p]$. The presence of categorical features is solved as standard through one-hot encoding just like in linear regression.
For **explainability**, the model remains transparent since the coefficients still indicate the strength and direction of the relationship between each feature and the target variable, but the interpretation is less intuitive than in linear regression due to the non-linear transformation applied to the output by the logistic function. An increase in one feature in logistic regression leads to a multiplicative change in the odds of the positive class, rather than an additive change in the predicted value.

3.2.5 Data assumptions

Before discussing the data assumptions we need to clarify the role of the odds. The odds of an event is defined as the ratio of the probability of the event occurring to the probability of it not occurring:

$$\text{odds} = \frac{P(y=1)}{P(y=0)} = \exp\left(\beta_0 + \sum_{j=1}^{p} \beta_j x_j\right)$$

This formulation more easily shows how the coefficients β_j affect the predictions.

$$\frac{\text{odds}_{x_j+1}}{\text{odds}} = \exp(\beta_j)$$

Just like in linear regression, the idea of maintaing the interpretability of the model through a linear combination of the features leads to several assumptions on the data and the model performance:

1. **Linearity of the log-odds:** $\log(\text{odds}) = \beta_0 + \sum_j \beta_j x_j$. The relationship is linear in the **log-odds space**, not in the probability space. Nonlinear relationships remain uncaptured and must be added manually.
2. **Complete separation:** if a feature perfectly separates the two classes , the model cannot identify a finite weight since the algorithm will tend to push the weight to $+\infty$ or $-\infty$, diverging. Usually the solution is to eraze this separating feature, since it is probably just a proxy for the target variable.
3. **Binomial distribution:** the response variable must follow a **Bernoulli distribution**.
4. **Independence of measurements:** observations should not be correlated. This means that the model is not suitable for temporal data without proper control, as well as for data with strong dependencies between observations.
5. **Fixed features:** features must be considered constants, without measurement error.
6. **Absence of multicollinearity:** highly correlated features cause coefficient instability. This is particularly relevant due to the fact that the iterative methods begin to oscillate and need very small learning rates (α).

Homoscedasticity is no more a requirment because the response can only be 0 or 1.

3.2.5.1 Preprocessing

To verify the suitability of logistic regression in the classification task for a given dataset, some methods can be used. The correlation matrix can be used to identify highly correlated features, which can lead to multicollinearity issues. If such features are found, they can be removed or combined to reduce redundancy and improve model stability.
For identifying anomalys in the other assumptions, plot visualitation can be used. See Section 3.3.1.10 for more details.

3.2.6 Predictive performance and limitations

The imposed assumptions of logistic regression can lead to good performance whenever this assumptions are met. The probabilistic interpretation gives insight on the confidence of the prediction, which is a significant advantage in many applications.
When the relationships are more complex than basic linear combinations, the predictions become less accurate. In context of unbalanced classes the training process can be dominated by the majority class, leading to poor performance on the minority class. Additionally, logistic regression as the linear regression is highly influenced by outliers, leading to very unstable predictions.

3.2.7 Punti di Forza

- **Output probabilistico:** diversamente da un classificatore duro, fornisce una stima di confidenza della previsione

- **Efficienza computazionale:** training veloce anche su dataset moderatamente grandi
- **Interpretabilità:** i pesi, sebbene moltiplicativi, rimangono interpretabili
- **Linearità decisionale:** il confine decisionale è una retta (in 2D) o iperpiano (in p-D)

3.2.8 Punti di Debolezza

- **Non cattura non linearità:** come LR, pattern non lineari complessi rimangono invisibili al modello
- **Sensibile a separazione completa:** diverge numericamente se esiste una feature "perfetta"
- **Interazioni nascoste:** effetti di feature che dipendono l'uno dall'altro rimangono invisibili

3.3 Metriche per la Confidenza

3.3.1 Metriche Pure

3.3.1.1 Confusion Matrix (Matrice di Confusione)

Confronta le classi reali con quelle predette, generando 4 categorie:

- **TP (True Positives):** previsione positiva corretta
- **TN (True Negatives):** previsione negativa corretta
- **FP (False Positives):** errore di tipo I, previsione positiva errata
- **FN (False Negatives):** errore di tipo II, previsione negativa errata

3.3.1.2 Accuracy

Frazione di previsioni corrette:

$$ACC = \frac{TP + TN}{TP + TN + FP + FN}$$

Limite: metrica misleading se le classi sono sbilanciate. Un classificatore che sempre predice la classe maggioritaria avrà accuracy alta

3.3.1.3 Sensitivity (Recall / True Positive Rate)

Frazione di positivi correttamente identificati:

$$Sens = \frac{TP}{TP + FN}$$

Risponde: "Su tutti i veri positivi, quanti abbiamo identificato?"

3.3.1.4 Specificity (True Negative Rate)

Frazione di negativi correttamente identificati:

$$Spec = \frac{TN}{TN + FP}$$

Risponde: "Su tutti i veri negativi, quanti abbiamo identificato?"

3.3.1.5 Precision

Frazione di previsioni positive corrette:

$$PREC = \frac{TP}{TP + FP}$$

Risponde: "Tra tutte le istanze che abbiamo predetto come positive, quante sono veramente positive?"

3.3.1.6 Recall

Alias per Sensitivity:

$$REC = \frac{TP}{TP + FN}$$

3.3.1.7 F1-Score

Media armonica tra Precision e Recall, utile quando si vuole bilanciare i due:

$$F1 = 2\frac{PREC \cdot REC}{PREC + REC}$$

Quando usarlo: quando le classi sono sbilanciate e sia falsi positivi che falsi negativi hanno costo significativo

3.3.1.8 ROC Curve e AUC

ROC Curve: rappresentazione grafica del trade-off tra True Positive Rate (Sensitivity) sull'asse y e False Positive Rate (1 - Specificity) sull'asse x, al variare del threshold di classificazione.

- **Modello ideale:** curva passa per il punto (0,1) in alto a sinistra
- **Modello casuale:** curva segue la diagonale (AUC = 0.5)

AUC (Area Under the Curve): area sotto la ROC curve, quantifica la capacità globale del modello di discriminare tra le classi

- **AUC = 1.0:** discriminazione perfetta
- **AUC = 0.5:** modello casuale
- **AUC < 0.5:** peggio del caso

AUC è **invariante al threshold**, quindi è preferibile ad Accuracy per valutazioni comparative.

3.3.1.9 Z-Statistic e p-value

Analogo della t-statistic in LR, misura la significatività statistica di ogni coefficiente:

$$Z = \frac{\beta_j}{SE(\beta_j)}$$

Dove $SE(\beta_j)$ è l'errore standard del coefficiente.

- **Valori assoluti grandi di Z** → forte evidenza che il coefficiente è significativamente diverso da 0
- **p-value associato** → probabilità di osservare Z così estremo sotto l'ipotesi nulla ($\beta_j = 0$)
- **Convention:** p < 0.05 indica significatività

3.3.1.10 Diagnostic plots

3.4 Metriche per la Comprensione e Spiegabilità

3.4.1 Effect Plot

Simile a LR, rappresenta l'effetto di una feature sulla **probabilità predetta** (non sugli odds, che è più difficile da interpretare).

Per ogni valore della feature, calcolare:

$$P(y = 1 \mid x_j = v, x_{\text{other}} = \text{media})$$

Il grafico mostra come la probabilità predetta varia col valore della feature, mantenendo le altre feature ai loro valori medi.

Vantaggio rispetto a LR: la trasformazione sigmoide rende visibile se l'effetto è principalmente presso certi valori della feature (grafico non è una retta, è una curva).

3.4.2 Weight Plot (Coefficienti)

Analogo a LR, rappresenta graficamente i coefficienti β_j ordinati per valore assoluto.

Caveat: il valore di β_j non corrisponde direttamente a una variazione di probabilità (è moltiplicativo sugli odds, non additivo sulla probabilità). Quindi il grafico ha **minore valore interpretativo** che in LR.

3.4.3 Odds Ratio

Espressione diretta dell'impatto di una feature sugli odds:

$$\text{OR}_j = \exp(\beta_j)$$

Interpretazione: "Aumentare feature j di 1 unità moltiplica gli odds per OR_j"

Esempio: se $\beta_j = 0.5$ e $\text{OR}_j = 1.65$, aumentare la feature del 65% gli odds di appartenenza alla classe positiva.

3.5 Limiti di Predizione

3.5.1 Non Linearità

Come LR, il modello non cattura pattern non lineari. La sigmoide è una trasformazione "accessoria" che non supera questo limite.

3.5.2 Separazione Completa

Se una feature separa perfettamente le classi, l'algoritmo di ottimizzazione **diverge numericamente**: il coefficiente tende a $+\infty$ o $-\infty$. Soluzioni:

- Penalizzare i pesi (Ridge o Lasso logistica)
- Rimuovere la feature (ma significa perdere informazione)
- Usare Firth's bias-reduced logistic regression (metodo specializzato)

3.5.3 Overfitting

Con un numero elevato di feature rispetto al numero di osservazioni ($p >> n$), il modello può facilmente sovradattarsi al training set. Ridge o Lasso logistica riducono il rischio.

3.5.4 Sensibilità al Classe Sbilanciate

Il modello di default è addestrato minimizzando il likelihood globale. Se una classe è molto rara (es. 1% positivi, 99% negativi), il modello tenderà a predire sempre la classe maggioritaria. Soluzioni:

- Usare pesi di classe inversi alla frequenza (class weights)
- Ricampionare (oversampling della classe rara o undersampling della classe maggioritaria)
- Regolare il threshold di classificazione (non usare 0.5 di default)

3.6 Limiti di Spiegabilità

3.6.1 Interpretazione Moltiplicativa Complessa

A differenza di LR dove un coefficiente corrisponde a una variazione additiva, qui $\exp(\beta_j)$ è moltiplicativo e non intuitivo per la maggior parte degli utenti. Una feature che aumenta gli odds del 65% non è semplice da comunicare rispetto a "la previsione aumenta di 10 unità".

3.6.2 Dipendenza dal Contesto

L'effetto di una feature sulla probabilità predetta **dipende dai valori di tutte le altre feature**. Per una feature, l'aumento della probabilità è maggiore quando le altre feature sono tali che la probabilità predetta è intorno a 0.5 (dove la sigmoid è più ripida). Questo rende difficile fornire una spiegazione "universale" dell'effetto di una feature.

3.6.3 Piccoli Coefficienti Difficili da Valutare

Se β_j è piccolo (es. 0.01), l'effetto $\exp(0.01) \approx 1.01$ (aumento del 1%) è difficile da discernere dalla variabilità naturale. Questo crea incertezza sull'importanza reale della feature.

3.6.4 Interazioni Nascoste

Se due feature hanno un effetto congiunto non additivo, il modello base non lo cattura. Le interazioni devono essere introdotte manualmente, il che aggiunge complessità nel comunicare i risultati.

3.7 Prompt

3.8 ...

Chapter 4

Design and code implementation

Breve introduzione al capitolo

4.1 Tecnologie e strumenti

Di seguito viene data una panoramica delle tecnologie e strumenti utilizzati.

4.1.1 Tecnologia 1

Descrizione Tecnologia 1.

4.1.2 Tecnologia 2

Descrizione Tecnologia 2

4.2 Progettazione

4.3 Design Pattern utilizzati

4.4 Codifica

Chapter 5

Prompt Engineering

Breve introduzione al capitolo

Chapter 6

Conclusion

6.1 Consuntivo finale

6.2 Raggiungimento degli obiettivi

6.3 Conoscenze acquisite

6.4 Valutazione personale

Bibliography

[1] Christoph Molnar, *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Leanpub, 2025.

[2] Zizhao Hu, Mohammad Rostami, and Jesse Thomason, "Expert Personas Improve LLM Alignment but Damage Accuracy: Bootstrapping Intent-Based Persona Routing with PRISM," 2026.

[3] "Which Table Format Do LLMs Understand Best?." [Online]. Available: https://www.improvingagents.com/blog/best-input-data-format-for-llms/

[4] Jasonn Wei *et al.*, "Chain of Thought Prompting Elicits Reasoning in Large Language Models," 2023.

[5] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa, "Large Language Models are Zero-Shot Reasoners," 2023.

[6] Taowen Wang *et al.*, "M^2PT: Multimodal Prompt Tuning for Zero-Shot Instruction Learning," 2024.

[7] "P-Value: What It Is, How to Calculate It, and Examples." [Online]. Available: https://www.investopedia.com/terms/p/p-value.asp

[8] Zhen "Leo" Liu, *Artificial Intelligence for Engineers: Basics and Implementations*. Springer, 2024.

[9] Candice Bentéjac, Anna Csörgő, and Gonzalo Martínez-Muñoz, "A Comparative Analysis of XGBoost," *Artificial Intelligence Review*, vol. 54, no. 3, 2019.

[10] Christopher M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[11] G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*. Springer, 2023.

[12] V. K. Xidong Wu, *Top 10 Algorithms in Data Mining*. CRC Press, 2009.

[13] Tianqi Chen and Carlos Guestrin, "XGBoost: A Scalable Tree Boosting System," *KDD '16*, 2016.

[14] Lior Rokach and Oded Maimon, *Data Mining with Decision Trees: Theory and Applications*. World Scientific Publishing, 2008.

[15] "SHAP Documentation." [Online]. Available: https://shap.readthedocs.io/en/latest/

[16] Xavier Amatriain, "Prompt Design and Engineering: Introduction and Advanced Methods," 2024.

Chapter 7

Glossary

***AI* – Artificial Intelligence**: Field of computer science that focuses on creating systems capable of performing tasks that typically require human intelligence, such as learning, reasoning, problem-solving, and understanding natural language. 1

***categorical features* – Categorical Features**: Features in a dataset that represent discrete categories or groups, rather than continuous numerical values, and often require special handling in machine learning algorithms. 4, 13, 15, 20

***classification* – Classification**: A type of machine learning problem in which the goal is to predict discrete class labels based on input features. i, 2

***computational complexity* – Computational complexity**: A measure of the amount of computational resources (time and space) that an algorithm requires to run, often expressed in terms of the size of the input data. 4, 12, 13, 13, 13, 13

***correlation matrix* – Correlation Matrix**: A table showing the correlation coefficients between pairs of features in a dataset, used to identify relationships and potential multicollinearity. 14, 21

***XAI* – Explainable Artificial Intelligence**: A field of artificial intelligence that focuses on creating models and systems that can provide clear and understandable explanations for their decisions and actions. i, i, 2, 9

***GD* – Gradient Descent**: An optimization algorithm used to minimize the objective function in machine learning models by iteratively adjusting the model's parameters in the direction of the steepest descent. 12, 13, 13, 14, 19, 19

***LLM* – Large Language Model**: A type of artificial intelligence model that is trained on vast amounts of text data to understand and generate human-like language. i, i, 2, 7, 7, 7, 9, 12

***MAE* – Mean Absolute Error**: A common metric for evaluating the performance of regression models, calculated as the average of the absolute differences between predicted and actual values. iv, 16

***ML* – Machine Learning**: A subset of artificial intelligence that involves training algorithms to learn patterns from data and make predictions or decisions without being explicitly programmed. i, 1

***objective_function* – Objective Function**: A mathematical function that an algorithm optimizes during training, which defines the goal of the learning process and guides the model's adjustments to minimize error or maximize performance. 4

outlier: A data point that deviates significantly from the majority of the data, which can have a disproportionate influence on machine learning models and may require special handling. 15

***PE* – Prompt Engineering**: The process of designing and optimizing prompts to effectively communicate with language models and elicit desired responses. 2

***regression* – Regression**: A type of machine learning problem in which the goal is to predict continuous numerical values based on input features. i, 2

***RMSE* – Root Mean Squared Error**: A common metric for evaluating the performance of regression models, calculated as the square root of the average of the squared differences between predicted and actual values. iv, 16

***SHAP* – SHapley Additive exPlanations**: A method for explaining the output of machine learning models by attributing the prediction to each feature. i

***SGD* – Stochastic Gradient Descent**: An optimization algorithm that updates the model's parameters using a randomly selected subset of the training data, which can lead to faster convergence and better generalization. 13, 20